Python - Remove Set Items

Remove Set Items

Removing set items implies deleting elements from a set. In Python, sets are mutable, unordered collections of unique elements, and there are several methods available to remove items from a set based on different criteria.

We can remove set items in Python using various methods such as remove(), discard(), pop(), clear(), and set comprehension. Each method provide different ways to eliminate elements from a set based on specific criteria or conditions.

Remove Set Item Using remove() Method

The remove() method in Python is used to remove the first occurrence of a specified item from a set.

We can remove set items using the remove() method by specifying the element we want to remove from the set. If the element is present in the set, it will be removed. However, if the element is not found, the remove() method will raise a KeyError exception.

Example

In the following example, we are deleting the element "Physics" from the set "my_set" using the remove() method –

```
my_set = {"Rohan", "Physics", 21, 69.75}
print ("Original set: ", my_set)
```

```
my_set.remove("Physics")
print ("Set after removing: ", my_set)
```

It will produce the following output –

```
Original set:  {21, 69.75, 'Rohan', 'Physics'}
Set after removing:  {21, 69.75, 'Rohan'}
```

Example

If the element to delete is not found in the set, the remove() method will raise a KeyError exception –

```
my_set = {"Rohan", "Physics", 21, 69.75}
print ("Original set: ", my_set)

my_set.remove("PHP")
print ("Set after removing: ", my_set)
```

We get the error as shown below –

```
Original set:  {'Physics', 21, 69.75, 'Rohan'}
Traceback (most recent call last):
  File "/home/cg/root/664c365ac1c3c/main.py", line 4, in <module>
    my_set.remove("PHP")
KeyError: 'PHP'
```

Remove Set Item Using discard() Method

The discard() method in set class is similar to remove() method. The only difference is, it doesn't raise error even if the object to be removed is not already present in the set collection.

Example

In this example, we are using the discard() method to delete an element from a set regardless of whether it is present or not –

```
my_set = {"Rohan", "Physics", 21, 69.75}
print ("Original set: ", my_set)

# removing an existing element
my_set.discard("Physics")
print ("Set after removing Physics: ", my_set)

# removing non-existing element
my_set.discard("PHP")
print ("Set after removing non-existent element PHP: ", my_set)
```

Following is the output of the above code −

```
Original set:  {21, 'Rohan', 69.75, 'Physics'}
Set after removing Physics:  {21, 'Rohan', 69.75}
Set after removing non-existent element PHP:  {21, 'Rohan', 69.75}
```

Remove Set Item Using pop() Method

We can also remove set items using the pop() method by removing and returning an arbitrary element from the set. If the set is empty, the pop() method will raise a KeyError exception.

Example

In the example below, we are defining a set with elements "1" through "5" and removing an arbitrary element from it using the pop() method −

```
# Defining a set
my_set = {1, 2, 3, 4, 5}

# removing and returning an arbitrary element from the set
removed_element = my_set.pop()

# Printing the removed element and the updated set
```

```
print("Removed Element:", removed_element)
print("Updated Set:", my_set)
```

We get the output as shown below −

```
Removed Element: 1
Updated Set: {2, 3, 4, 5}
```

Example

If we try to remove element from an empty set, the pop() method will raise a KeyError exception −

```
# Defining an empty set
empty_set = set()

# Removing an arbitrary element from the empty set
removed_element = empty_set.pop()
```

The error produced is as shown below −

```
Traceback (most recent call last):
  File "/home/cg/root/664c69620cd40/main.py", line 5, in <module>
removed_element = empty_set.pop()
KeyError: 'pop from an empty set'
```

Remove Set Item Using clear() Method

The clear() method in set class removes all the items in a set object, leaving an empty set.

We can remove set items using the clear() method by removing all elements from the set, effectively making it empty.

Example

In the following example, we are defining a set with elements "1" through "5" and then using the clear() method to remove all elements from the set −

```
# Defining a set with multiple elements
my_set = {1, 2, 3, 4, 5}

# Removing all elements from the set
my_set.clear()
# Printing the updated set
print("Updated Set:", my_set)
```

It will produce the following output −

```
Updated Set: set()
```

Remove Items Existing in Both Sets

You can remove items that exist in both sets (i.e., the intersection of two sets) using the difference_update() method or the subtraction operator (-=). This removes all elements that are present in both sets from the original set.

Example

In this example, we are defining two sets "s1" and "s2", and then using the difference_update() method to remove elements from "s1" that are also in "s2" −

```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}
print ("s1 before running difference_update: ", s1)
s1.difference_update(s2)
print ("s1 after running difference_update: ", s1)
```

After executing the above code, we get the following output −

```
s1 before running difference_update: {1, 2, 3, 4, 5}
s1 after running difference_update: {1, 2, 3}
set()
```

Remove Items Existing in Either of the Sets

To remove items that exist in either of two sets, you can use the symmetric difference operation. The symmetric difference between two sets results in a set containing elements that are in either of the sets but not in their intersection.

In Python, the symmetric difference operation can be performed using the **^** operator or symmetric_difference() method.

Example

In the following example, we are defining two sets "set1" and "set2". We are then using the symmetric difference operator (^) to create a new set "result_set" containing elements that are in either "set1" or "set2" but not in both −

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

# Removing items that exist in either set
result_set = set1 ^ set2
print("Resulting Set:", result_set)
```

Following is the output of the above code −

```
Resulting Set: {1, 2, 5, 6}
```

Remove Uncommon Set Items

You can remove uncommon items between two sets using the intersection_update() method. The intersection of two sets results in a set containing only the elements that are present in both sets.

To keep only the common elements in one of the original sets and remove the uncommon ones, you can update the set with its intersection.

Example

In this example, we are defining two sets "set1" and "set2". We are then using the intersection_update() method to modify "set1" so that it only contains elements that are also in "set2" −

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

# Keeping only common items in set1
set1.intersection_update(set2)
print("Set 1 after keeping only common items:", set1)
```

Output of the above code is as shown below −

```
Set 1 after keeping only common items: {3, 4}
```

The intersection() Method

The intersection() method in set class is similar to its intersection_update() method, except that it returns a new set object that consists of items common to existing sets.

Syntax

Following is the basic syntax of the intersection() method −

```
set.intersection(obj)
```

Where, **obj** is a set object.

Return value

The intersection() method returns a set object, retaining only those items common in itself and obj.

Example

In the following example, we are defining two sets "s1" and "s2", then using the intersection() method to create a new set "s3" containing elements that are common to both "s1" and "s2" −

```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}
print ("s1: ", s1, "s2: ", s2)
s3 = s1.intersection(s2)
print ("s3 = s1 & s2: ", s3)
```

It will produce the following output −

```
s1: {1, 2, 3, 4, 5} s2: {4, 5, 6, 7, 8}
s3 = s1 & s2: {4, 5}
```

Symmetric Difference Update of Set Items

The symmetric difference between two sets is the collection of all the uncommon items, rejecting the common elements. The symmetric_difference_update() method updates a set with symmetric difference between itself and the set given as argument.

Example

In the example below, we are defining two sets "s1" and "s2", then using the symmetric_difference_update() method to modify "s1" so that it contains elements that are in either "s1" or "s2", but not in both −

```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}
print ("s1: ", s1, "s2: ", s2)
s1.symmetric_difference_update(s2)
print ("s1 after running symmetric difference ", s1)
```

The result obtained is as shown below −

```
s1: {1, 2, 3, 4, 5} s2: {4, 5, 6, 7, 8}
s1 after running symmetric difference {1, 2, 3, 6, 7, 8}
```

Symmetric Difference of Set Items

The symmetric_difference() method in set class is similar to symmetric_difference_update() method, except that it returns a new set object that holds all the items from two sets minus the common items.

Example

In the following example, we are defining two sets "s1" and "s2". We are then using the symmetric_difference() method to create a new set "s3" containing elements that are in either "s1" or "s2", but not in both −

```
s1 = {1,2,3,4,5}
s2 = {4,5,6,7,8}
print ("s1: ", s1, "s2: ", s2)
s3 = s1.symmetric_difference(s2)
print ("s1 = s1^s2 ", s3)
```

It will produce the following output −

```
s1: {1, 2, 3, 4, 5} s2: {4, 5, 6, 7, 8}
s1 = s1^s2 {1, 2, 3, 6, 7, 8}
```

TRENDING TECHNOLOGIES

Cloud Computing Tutorial

Amazon Web Services Tutorial

Microsoft Azure Tutorial

Git Tutorial

Ethical Hacking Tutorial

Docker Tutorial

Kubernetes Tutorial

DSA Tutorial

Spring Boot Tutorial

SDLC Tutorial

Unix Tutorial

CERTIFICATIONS

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler

Online Bash Terminal

Online SQL Compiler

Online Html Editor